

**ФУНКЦІОН
ПАРАДИ
ПРОГРАМУ
РУТНО**

ВСТУП

- Досі: об'єкти, класи, мутабельні
- Сьогодні: обчислення без побіч
- План: від математики до практи

1. МАТЕМАТИЧЕСКАЯ БАЗИ

**ЛЯМБДА-ЧИСЛЕН
ЧЕРЧА**

ЛЯМБДА-ЧИСЛЕН ЧЕРЧА

- 1930-ті, паралельно з машиною

ЛЯМБДА-ЧИСЛЕННЯ ЧЕРЧА

- 1930-ті, паралельно з машиною
- Обчислення = застосування фу

ЛЯМБДА-ЧИСЛЕННЯ ЧЕРЧА

- 1930-ті, паралельно з машиною
- Обчислення = застосування фу
- Теза Черча–Тюрінга: еквівалент

НОТАЦІЯ ЛЯМБДА-ЧИСЛЕ

$\lambda x. x + 1$	— визначення функції
$(\lambda x. x + 1) 5$	— застосування, результат

Y PYTHON

```
f = lambda x: x + 1  
f(5)    # 6
```

ЧИСТІ ФУНКЦІЇ (PURE

- Детермінованість: той самий вх
- Відсутність побічних ефектів
- Тестованість, кешованість, без

```
total = 0
def add_dirty(x):
    global total
    total += x
    return total

def add_pure(x, y):
    return x + y
```

ПОБІЧНІ ЕФЕКТИ — ЦЕ

- Зміна глобальних змінних
- Мутація переданих аргументів
- I/O: файли, консоль, мережа, БД
- `random()`, поточний час

НЕЗМІННІСТЬ (IMMUTABILITY)

- Незмінні типи: `int`, `str`, `tuple`,
- Змінні типи: `list`, `dict`, `set`
- Замість зміни — створення нов

```
point = (10, 20)
point[0] = 15    # TypeError

new_point = (point[0] + 5, point[1])
```

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Point:
    x: int
    y: int

p1 = Point(10, 20)
p1.x = 15      # FrozenInstanceError
```

2. МЕХАНИКА

ФУНКЦІЇ ПЕРШОГО І CLASS CITIES

- Функції — об'єкти: присвоєння
- Функції вищого порядку (Higher-Order Functions)

```
funcs = [str.upper, str.lower, str.  
  
def apply_twice(func, x):  
    return func(func(x))  
  
apply_twice(lambda x: x * 2, 5)    #
```

ЛЯМБДА-ВИРАЗИ: ОБ

- Лише **один вираз**, не блок коду
- Заборонено: присвоєння, `for`,
- Дозволено: тернарний вираз

```
classify = lambda x: "додатне" if x  
sorted(students, key=lambda s: s.gr
```

ЗАМИКАННЯ (CLOSURE)

- Внутрішня функція "пам'ятає" scope
- nonLocal для зміни захопленої змінної

```
def make_multiplier(n):  
    def multiplier(x):  
        return x * n  
    return multiplier  
  
double = make_multiplier(2)  
double(5)    # 10
```

```
def make_counter():  
    count = 0  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
  
counter = make_counter()
```

ПРОМІЖНИЙ ПІДСУМ

- Черч → чисті функції → незмінні
- First-class functions → lambda –

Python

Далі: конвеєри обробки даних 🙌

3. КОНВЕЕР ОПТИМІЗАЦІЯ

MAP, FILTER, REDUCE

```
from functools import reduce

data = [1, 2, 3, 4, 5, 6, 7, 8]

doubled = map(lambda x: x * 2, data)
evens    = filter(lambda x: x % 2 == 0, data)
total    = reduce(lambda a, b: a + b, evens)
```

ІДІОМАТИЧНИЙ ВАРІАНТ COMPREHENSION

```
total = sum(x * 2 for x in data if
```

ЧАСТКОБЕ ЗАСТ (PARTIAL APPL

```
from functools import partial

def power(base, exponent):
    return base ** exponent

square = partial(power, exponent=2)
cube    = partial(power, exponent=3)

square(5)    # 25
```

КАРРИРУВАННЯ (CURRY)

```
def curry_add(x):  
    def inner(y):  
        return x + y  
    return inner  
  
add5 = curry_add(5)  
add5(10)          # 15  
curry_add(5)(10)  # 15
```

РЕКУРСІЯ ТА СТЕК БАЗА

- Рекурсія: функція викликає саму себе
- Кожен виклик створює новий кадр
- Кадр зберігає: локальні змінні - функція
- Стек обмежений → занадто глибокий → переповнення

ПРИКЛАД: ЗВИЧАЙНА РЕКУРС

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)    #
```

ЩО ВІДБУВАЄТЬСЯ FACTORIA

```
factorial(4)
  → factorial(3)
    → factorial(2)
      → factorial(1) = 1
        ← 1,    МНОЖИМО на 2 → 2
      ← 2,    МНОЖИМО на 3 → 6
    ← 6,    МНОЖИМО на 4 → 24
```

- Усі 4 кадри одночасно "висять" результат вкладеного виклику

ХВОСТОВИЙ ВИКЛИК ЩО ЦЕ

- Рекурсивний виклик — це **оста**
- З результатом виклику більше і одразу повертається далі
- Отже, кадр поточного виклику

```
def factorial_tail(n, acc=1):  
    if n <= 1:  
        return acc  
    return factorial_tail(n - 1, n * acc)
```

- `acc` накопичує результат заздалегідь відкладеного множення при по

ІДЕЯ TCO (TAIL CALL OPTIMIZATION)

- Якщо виклик хвостовий — можливий видалити найнижчий кадр стека замість створення нового
- Рекурсія за пам'яттю стає схожою на цикл
- Підтримується у Scheme, Haskell, Erlang, JavaScript, але не в Python

PYTHON СВІДОМО НЕ

- Це не технічне обмеження — пр
- ван Россума
- ТСО "стирає" проміжні кадри – при помилці
- Філософія Python: "явне краще важливіший за оптимізацію
- Наслідок: `RecursionError` після `(sys.getrecursionlimit())`

СИМУЛЯЦІЯ БЕЗ ТСО

- Акумулятор (як `factorial_tail`)
але стек все одно росте в Python
- Явний цикл — найнадійніший сп

ЛІНИВІ ОБЧИСЛЕННЯ EVALUATION

- `map/filter` уже лінійні за замов
- Генератори: `yield` та `generator`

```
def lazy_pipeline(data):  
    for x in data:  
        d = x * 2  
        if d % 2 == 0:  
            yield d
```

```
pipeline = (x * 2 for x in data if
```

ПАМ'ЯТЬ: СПИСОК VS ГЕН

```
import sys

data = range(1_000_000)
sys.getsizeof([x * 2 for x in data])
sys.getsizeof((x * 2 for x in data))
```

ПРАКТИКА: НЕСКІНЧ

```
from itertools import islice

def natural_numbers():
    n = 1
    while True:
        yield n
        n += 1

def doubled_evens(nums):
```

ПАТЕРН-МАТЧИНГ

PATTERN MATCHING

- Python 3.10+, оператор `match` / `case`
- У функціональних мовах (Haskell) стандартний спосіб розгалуження
- Найкраще розкривається саме

БАЗОВИЙ СИНТАКСИС

```
def describe(value):  
    match value:  
        case 0:  
            return "нуль"  
        case n if n > 0:           # guaranteed  
            return "додатне"  
        case _:                   # impossible  
            return "від'ємне"
```

РОЗБІР СТРУКТУР (DECON

```
match point:
    case (0, 0):
        print("початок координат")
    case (x, 0):
        print(f"на осі X: {x}")
    case (x, y):
        print(f"точка ({x}, {y})")
```

- case не лише порівнює значення структури одразу в змінні

МАТЧИНГ КЛАСІВ

```
@dataclass(frozen=True)
class Point:
    x: int
    y: int

match p:
    case Point(x=0, y=0):
        print("початок координат")
    case Point(x=x, y=y) if x == y:
```

ЧОМУ ЦЕ ПОРУЧ ІЗ ФУНКЦІОНАЛЬНИМ СТИЛЕМ

- Замінює довгі ланцюжки `if/else` формою
- Природний партнер для `frozen` попередніх лекцій
- В ФП-мовах `pattern matching` — працювати з алгебраїчними типами його концептуальний родич, хоча

ПІДСУМОК КУРСУ

- Чисті функції + незмінність → **НЕЗМІННОСТЬ**
 - HOF + замикання → **КОМПОЗИЦІЙНОСТЬ**
 - Map/filter/reduce + partial/curry → **КАПСУЛІЗАЦІЯ**
 - Генератори → **МАСШТАБОВАНІСТЬ**
 - Pattern matching → **ДЕКЛАРАТИВНОСТЬ**
- ДАНИХ**

ДЯКУЮ ЗА УВАГУ!